

Pseudocode Fundamentals

Pseudocode is an informal high-level description of the operating principle of a computer program or other algorithm. It uses the structural conventions of a normal programming language, but is intended for human reading rather than machine reading.

Key Characteristics

- *Language-Independent*: Not tied to any specific programming language.
- *Step-by-Step Logic*: Focuses on the sequence of operations.
- *Readability*: Uses simple, clear English-like statements.
- *Abstraction*: Hides low-level details of implementation.

Core Building Blocks

All algorithms, no matter how complex, are built from these three fundamental structures:

1. **Sequence**: Step-by-step execution of instructions.
2. **Selection (Decision Making)**: Executing different blocks of code based on a condition (IF-ELSE).
3. **Iteration (Repetition/Loops)**: Repeating a block of code multiple times (FOR, WHILE).

Building Block Summary

Building Block	Description	Example Pseudocode
Sequence	Instructions executed one after another.	READ A, B; SUM := A + B; PRINT SUM
Selection	Choosing between different paths based on a condition.	IF A > B THEN PRINT A ELSE PRINT B END IF
Iteration	Repeating a set of instructions.	FOR i FROM 1 TO 5 PRINT i END FOR

Common Pseudocode Keywords & Syntax

Variables & Assignment

- SET Variable := Value or Variable := Value
- READ Variable: Input from user/source.
- PRINT Expression: Output to screen/destination.

Conditional Statements (Selection)

IF condition THEN // action 1 ELSE // action 2 END IF
IF condition1 THEN // action 1 ELSE IF condition2 THEN // action 2 ELSE // action 3 END IF

Looping Constructs (Iteration)

- **FOR Loop**:

FOR i FROM start_value TO end_value [STEP step_value]
 // loop body
ENDFOR
- **WHILE Loop**:

WHILE condition
 // loop body
ENDWHILE
- **BREAK**: Exits the innermost loop immediately.

Arithmetic Operators

+ (Addition), - (Subtraction), * (Multiplication), / (Division), MOD (Modulo - remainder of division), DIV (Integer Division - quotient).

Relational Operators

= (Equal to), != or <> (Not equal to), < (Less than), > (Greater than), <= (Less than or equal to), >= (Greater than or equal to).

Logical Operators

AND, OR, NOT.

String Operations

- LENGTH(String): Returns the length of the string.
- SUBSTRING(String, Start, Length): Extracts a part of the string. E.g., SUBSTRING("Hello", 2, 3) returns "ell".
- CONCAT(String1, String2): Joins two strings.
- TO_UPPERCASE(String), TO_LOWERCASE(String).
- CH IS A LETTER, CH IS A DIGIT, CH IS A VOWEL, etc.
- ORD(Character): Returns the ASCII/Unicode integer value of a character. E.g., ORD('A') = 65.
- CHR(Integer): Returns the character corresponding to an ASCII/Unicode integer value. E.g., CHR(66) = 'B'.

Dry Run / Trace Tables

A crucial technique for understanding and debugging pseudocode is to perform a "dry run" using a trace table. This involves manually tracking the value of each variable step-by-step.

Example: Even-Odd Challenge Trace

```
A := 1
FOR I FROM 1 TO 6
  IF i MOD 2 = 0 THEN
    A := A * i
  ELSE
    A := A + i
  END IF
ENDFOR
PRINT A
```

	i	i MOD 2	Condition	A (before)	Operation	New A
Initial	-	-	-	1	-	-
	1	1	Odd	1	A+1	2
	2	0	Even	2	A*2	4
	3	1	Odd	4	A+3	7
	4	0	Even	7	A*4	28
	5	1	Odd	28	A+5	33
	6	0	Even	33	A*6	198
Final A						198

Common Algorithmic Patterns

1. Sum of N Numbers

```
READ N
SUM := 0
FOR I FROM 1 TO N
  SUM := SUM + i
ENDFOR
PRINT SUM
```

--	--	--	--	--	--

2. Factorial

```
READ N
FACT := 1
FOR I FROM 1 TO N
  FACT := FACT * i
ENDFOR
PRINT FACT
```

--	--	--	--	--	--

3. Reverse a Number

```
READ N
REV := 0
WHILE N > 0
  DIGIT := N MOD 10
  REV := (REV * 10) + DIGIT
  N := N DIV 10
ENDWHILE
PRINT REV
```

--	--	--	--	--	--

Example Trace for Reverse (N=916)

N (before)	REV (before)	DIGIT	REV (after)	N (after)
916	0	-	-	-
916	0	916 MOD 10 = 6	0 * 10 + 6 = 6	916 DIV 10 = 91
91	6	91 MOD 10 = 1	6 * 10 + 1 = 61	91 DIV 10 = 9
9	61	9 MOD 10 = 9	61 * 10 + 9 = 619	9 DIV 10 = 0
Final REV				619

Advanced Concepts

Accumulator Reset

A variable (TEMP) might be used to accumulate a value temporarily and then reset based on a condition, contributing its value to a main accumulator (TOTAL).

```
SET TOTAL := 0
SET TEMP := 0
FOR I FROM 1 TO 4
  TEMP := I
  IF TEMP MOD 2 = 0 THEN
    TOTAL := TOTAL + TEMP
  TEMP := 0 // Reset TEMP for even numbers
  END IF
ENDFOR
PRINT TOTAL + TEMP // Add any remaining TEMP value
```

--	--	--	--	--	--

Trace for Accumulator Reset

I	TEMP (start)	TEMP MOD 2	Condition	TOTAL (after)	TEMP (after)
-	0	-	-	0	0
1	1	1	False	0	1
2	2	0	True	0 + 2 = 2	0
3	3	1	False	2	3
4	4	0	True	2 + 4 = 6	0
Final Output:				TOTAL + TEMP = 6 + 0 = 6	

Nested Loops with Conditionals

When loops are nested, the inner loop completes all its iterations for each iteration of the outer loop. Conditionals inside can modify variables based on both loop counters.

```
SET A := 2
FOR I FROM 1 TO 3
  FOR J FROM 1 TO 1
    IF (I + J) MOD 2 = 0 THEN
      A := A + 1
    ELSE
      A := A - 1
    END IF
  ENDFOR
ENDFOR
PRINT A
```

--	--	--	--	--	--

Trace for Nested Loops

I	J	(I+J)%2	Condition	A (before)	A (after)
-	-	-	-	2	-
1	1	2	0	True	2 + 1 = 3
2	1	3	1	False	3 - 1 = 2
2	2	4	0	True	2 + 1 = 3
3	1	4	0	True	3 + 1 = 4
3	2	5	1	False	4 - 1 = 3
3	3	6	0	True	3 + 1 = 4
Final A					4

Recursion

A function that calls itself.

Essential components

- **Base Case**: Condition that stops recursion, preventing infinite loops.
- **Recursive Step**: The function calls itself with modified arguments, moving towards the base case.

Example: Factorial (Recursive)

```
FUNCTION Factorial(n)
  IF n = 0 THEN
    RETURN 1
  ELSE
    RETURN n * Factorial(n - 1)
  END IF
END FUNCTION
```

--	--	--	--	--	--

Example: Fibonacci Sequence (Recursive)

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1.

```
FUNCTION Fibonacci(n)
  IF n <= 1 THEN
    RETURN n
  ELSE
    RETURN Fibonacci(n - 1) + Fibonacci(n - 2)
  END IF
END FUNCTION
```

--	--	--	--	--	--

Fibonacci(4) trace

```
// Fibonacci(4)
// = Fibonacci(3) + Fibonacci(2)
// = (Fibonacci(2) + Fibonacci(1)) + (Fibonacci(1) + Fibonacci(0))
// = ((Fibonacci(1) + Fibonacci(0)) + 1) + (1 + 0)
// = ((1 + 0) + 1) + (1)
// = (1 + 1) + 1
// = 2 + 1
// = 3
// Output: 3
```

--	--	--	--	--	--

Example: Sum of Digits (Recursive)

```
FUNCTION mystery(x) // Sum of digits
  IF x < 10 THEN
    RETURN x
  ELSE
    RETURN (x MOD 10) + mystery(x DIV 10)
  END IF
END FUNCTION
```

--	--	--	--	--	--

mystery(41) trace

```
// mystery(41) = (41 MOD 10) + mystery(41 DIV 10)
// = 1 + mystery(4)
// mystery(4) = 4 (base case: x < 10)
// Result = 1 + 4 = 5
```

--	--	--	--	--	--

Mutual Recursion

Two or more functions call each other in a cyclical manner.

```
FUNCTION funA(x)
  IF x <= 0 THEN RETURN 0
  ELSE RETURN x + funB(x - 1)
  END IF
END FUNCTION

FUNCTION funB(y)
  IF y <= 0 THEN RETURN 0
  ELSE RETURN y + funA(y - 2)
  END IF
END FUNCTION
```

--	--	--	--	--	--

PRINT funA(4) trace

```
// PRINT funA(4)
// funA(4) = 4 + funB(3)
// funB(3) = 3 + funA(1)
// funA(1) = 1 + funB(0)
// funB(0) = 0 (base case)
// funA(1) = 1 + 0 = 1
// funB(3) = 3 + 1 = 4
// funA(4) = 4 + 4 = 8
// Output: 8
```

Array Operations

Arrays are collections of elements, often accessed by an index (usually 1-based in pseudocode unless specified).

Accessing Elements

```
arr[index]
```

Iterating through Arrays

```
FOR I FROM 1 TO N
  READ arr[i]
ENDFOR
```

Example: Sum of Alternate Elements

```
READ N
SUM := 0
FOR I FROM 1 TO N
  READ arr[i]
  IF i MOD 2 = 1 THEN // Check if index is odd
    SUM := SUM + arr[i]
  END IF
ENDFOR
PRINT SUM
```

Example: Array Transformation (Recursive)

```
FUNCTION transform(arr, idx, N)
  IF idx > N THEN RETURN
  END IF
  arr[idx] := arr[idx] * idx
  transform(arr, idx + 1, N)
END FUNCTION
```

Initial array: arr = [3, 5, 2, 7], N = 4

```
// transform(arr, 1, 4)
// idx=1: arr[1] = 3 * 1 = 3
// idx=2: arr[2] = 5 * 2 = 10
// idx=3: arr[3] = 2 * 3 = 6
// idx=4: arr[4] = 7 * 4 = 28
// Final arr = [3, 10, 6, 28]
```

Bitwise Operations

- AND: Bitwise AND
- OR: Bitwise OR
- XOR: Bitwise Exclusive OR (returns 1 if bits are different, 0 if same)
- NOT: Bitwise NOT (flips bits)
- << (Left Shift), >> (Right Shift)

Key Property of XOR

- $X \oplus X = 0$
- $X \oplus 0 = X$

Example: Bitwise Array Puzzle (Find unique element in array where all others appear twice)

```
READ N
READ ARR[1..N]
SET result := 0
FOR I FROM 1 TO N
  result := result XOR arr[i]
ENDFOR
PRINT result
```

If ARR = [2, 3, 1, 2, 1]

```
// result = 0 XOR 2 XOR 3 XOR 1 XOR 2 XOR 1
//           = (2 XOR 2) XOR (1 XOR 1) XOR 3
//           = 0 XOR 0 XOR 3
//           = 3
// Output: 3
```

Example: Counting Set Bits (Number of 1s in binary representation)

```
READ N
SET COUNT := 0
WHILE N > 0
  IF N MOD 2 = 1 THEN // Check if the last bit is 1
    COUNT := COUNT + 1
  END IF
  N := N DIV 2 // Right shift (remove last bit)
ENDWHILE
PRINT COUNT
```

For N = 13 (binary 1101)

```
// N=13, MOD=1, COUNT=1, N=6
// N=6, MOD=0, COUNT=1, N=3
// N=3, MOD=1, COUNT=2, N=1
// N=1, MOD=1, COUNT=3, N=0
// Output: 3
```

Pattern Printing

Uses nested loops and conditional statements to print characters or numbers in specific patterns.

Example: Cross Pattern

```
READ N //e.g., N=5
FOR I FROM 1 TO N
  FOR J FROM 1 TO N
    IF (i = j) OR (i + j = N + 1) THEN
      PRINT "*"
    ELSE
      PRINT "."
    END IF
  ENDFOR
  PRINT NEWLINE
ENDFOR
```

Debugging & Tracing Complex Logic

Break Down Complex Problems

If the pseudocode is long or intricate, try to identify smaller, independent blocks of logic (e.g., a function, a nested loop) and understand what each block does in isolation.

Recognize Patterns

- **Accumulators:** Variables like SUM := SUM + X or PRODUCT := P RODUCT * X.
- **Counters:** Variables like COUNT := COUNT + 1.
- **Flags:** Boolean variables used to track a state (e.g., SET FOUND := TRUE).
- **Swapping:** Identify code that exchanges values between two variables.
- **Function Calls:** When a function is called, mentally (or physically on your scratchpad) pause the current execution, jump to the function definition, trace its execution, and then return to the calling point with the returned value.
- **Bitwise Operations:** Convert numbers to binary if bitwise operations are involved, then perform the operation bit by bit.
- **String Manipulation:** Pay close attention to start and length parameters for substring operations, and ASCII values for character conversions.

Increment/Decrement Operators

- **Pre-increment/decrement** ($++X$ or $--X$): The value of the variable is changed *before* it is used in the expression. Example: $Y := ++X$ means $X := X + 1$; $Y := X$;
- **Post-increment/decrement** ($X++$ or $X--$): The value of the variable is changed *after* it is used in the expression. Example: $Y := X++$ means $Y := X$; $X := X + 1$;

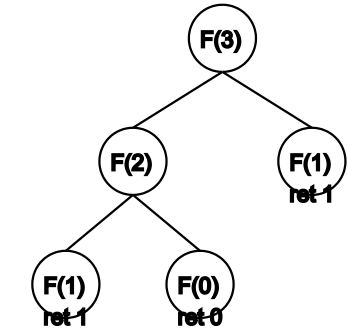
Always trace these operations carefully, as the order of evaluation is critical.

Cracking Recursive Problems (Fibonacci-like)

- **Identify the Base Case(s):** These are the conditions that stop the recursion. For Fibonacci, it's typically when $n = 0$ or $n = 1$. These are the known starting points.
- **Understand the Recursive Step:** How does the problem break down into smaller, similar subproblems? For Fibonacci, $F(n) = F(n - 1) + F(n - 2)$.
- **Draw a Recursion Tree:** This is extremely helpful for visualizing the flow of execution and understanding how many times each subproblem is called.
 - Start with the initial call (e.g., Fibonacci(4)).
 - Branch out for each recursive call.
 - Stop branching when you hit a base case and write down its return value.
 - Work your way back up the tree, calculating results.
- **Trace the Call Stack:** If a tree is too complex, imagine a stack where each function call is pushed, and pops off when it returns a value. This helps manage the state.
- **Memoization (if allowed/relevant):** For competitive programming, Fibonacci is often solved with memoization (storing results of subproblems to avoid recomputing), but in pseudocode tracing, you're usually expected to follow the direct recursive calls.

Recursion Tree Drawing Technique

Let's illustrate with Fibonacci(3) from the example above:



Interpretation

- Each circle represents a function call.
- Arrows indicate calls to subproblems.
- "ret X" indicates the return value from a base case or a computed value.
- Work from bottom-up to compute the final result.

Key Steps for Tracing

Always use a dry run table, especially for:

- Nested conditions (IF-ELSE inside IF-ELSE)
- Nested loops
- Variables whose values change within a loop and affect loop conditions or other calculations.
- Recursion: Trace function calls and returns meticulously.

1. Initialize all variables.
2. For each iteration of a loop or call to a function:
 - Record the current values of all relevant variables.
 - Evaluate conditions.
 - Execute the appropriate code block.
 - Update variable values.
 - Record any output.
3. The final state of variables and the complete output sequence will be your answer.

Tips for Infosys Pseudocode Round

Before Approaching a Question

- **Read Carefully:** Understand the problem statement, inputs, outputs, and any constraints.
- **Identify Keywords:** Look for READ, PRINT, SET, IF/ELSE, FOR/ WHILE, FUNCTION, and operators (MOD, DIV, AND, OR, XOR).
- **Determine Data Types:** Integer, String, Boolean, Array. Be aware of how operations like division work with integers.
- **Check Indexing:** For arrays, determine if indexing starts from 0 or 1. Most pseudocode tends to be 1-based, but always verify.
- **Look for Exit Conditions:** For loops (WHILE, FOR) and recursive functions (base cases), understand when they terminate.

Techniques to Solve

- **Dry Run is King:** For almost every question, especially those with loops, conditionals, or recursion, a dry run with a trace table is the most reliable method.
- Use small, representative input values for your dry run.
- Create columns for each variable that changes, loop counters, and conditions.
- Step through the code line by line, updating values.

Other Types of Questions & Techniques

Beyond the basics, Infosys pseudocode rounds may include:

1. Data Structure Manipulations

- **Arrays/Lists:** Sorting (Bubble, Selection, Insertion - though not explicitly coded, understanding their logic helps), searching (Linear, Binary), reversing, finding min/max, frequency counts.
- **Matrices (2D Arrays):** Traversing rows/columns, diagonal sums, specific pattern checks.

Example: Binary Search (Array must be sorted)

```
FUNCTION BinarySearch(arr, target, low, high)
  IF low > high THEN
    RETURN -1 // Not found
  END IF
  MID := (low + high) DIV 2
  IF arr[MID] = target THEN
    RETURN MID // Found at index MID
  ELSE IF arr[MID] < target THEN
    RETURN BinarySearch(arr, target, MID + 1, high)
  ELSE
    RETURN BinarySearch(arr, target, low, MID - 1)
  END IF
END FUNCTION
```

2. Number Theory Problems

Prime number checking, finding GCD/LCM, digit manipulation (sum of digits, reverse, count digits).

Example: Check if a number is Prime

```
FUNCTION IsPrime(N)
  IF N <= 1 THEN RETURN FALSE
  IF N <= 3 THEN RETURN TRUE // 2 and 3 are prime
  IF N MOD 2 = 0 OR N MOD 3 = 0 THEN RETURN FALSE
  i := 5
  WHILE i * i <= N
    IF N MOD i = 0 OR N MOD (i + 2) = 0 THEN RETURN FALSE
    i := i + 6
  END WHILE
  RETURN TRUE
END FUNCTION
```

3. String Processing

Palindrome check, counting vowels/consonants, character manipulation (case conversion, ASCII values).

Example: Palindrome Check

```
FUNCTION IsPalindrome(S)
  LEFT := 1
  RIGHT := LENGTH(S)
  WHILE LEFT < RIGHT
    IF S[LEFT] != S[RIGHT] THEN
      RETURN FALSE
    END IF
    LEFT := LEFT + 1
    RIGHT := RIGHT - 1
  END WHILE
  RETURN TRUE
END FUNCTION
```

4. Simulation Problems

Problems that simulate a process step-by-step, often involving state changes and specific rules. These heavily rely on accurate dry runs.

Example: Game of Life (Simplified Cell State Change)

Assume GRID is a 2D array of 0s and 1s. A cell flips if it has exactly 2 live neighbors.

```
FUNCTION UpdateCell(GRID, R, C, MAX_R, MAX_C)
  LIVE_NEIGHBORS := 0
  // Check 8 neighbors (simplified)
  FOR DR FROM -1 TO 1
    FOR DC FROM -1 TO 1
      IF DR = 0 AND DC = 0 THEN CONTINUE
      NR := R + DR
      NC := C + DC
      IF NR >= 1 AND NR <= MAX_R AND NC >= 1 AND NC <= MAX_C THEN
        IF GRID[NR][NC] = 1 THEN
          LIVE_NEIGHBORS := LIVE_NEIGHBORS + 1
        END IF
      END IF
    END FOR
  END FOR
  IF GRID[R][C] = 1 AND LIVE_NEIGHBORS = 2 THEN
    RETURN 0 // Dies
  ELSE IF GRID[R][C] = 0 AND LIVE_NEIGHBORS = 2 THEN
    RETURN 1 // Comes to life
  ELSE
    RETURN GRID[R][C] // Stays same
  END IF
END FUNCTION
```

5. Logic Puzzles / Mathematical Series

Problems involving sequences, series, or specific mathematical properties.

Example: Print a modified series

Given N, print N terms of series: 1, 1, 2, 3, 5, ... but if a term is even, print its half.

```
FUNCTION ModifiedFibonacciSeries(N)
  IF N <= 0 THEN RETURN
  a := 0
  b := 1
  IF N >= 1 THEN
    PRINT 1 // First term is always 1 in this problem for N>=1
  END IF
  FOR I FROM 2 TO N
    next_term := a + b
    IF next_term MOD 2 = 0 THEN
      PRINT next_term DIV 2
    ELSE
      PRINT next_term
    END IF
    a := b
    b := next_term
  END FOR
END FUNCTION
```

6. Function Parameter Passing

- **Pass by Value:** A copy of the argument's value is passed to the function. Changes inside the function do not affect the original variable outside.
- **Pass by Reference:** A reference (memory address) of the argument is passed. Changes inside the function directly affect the original variable outside. Pseudocode often uses keywords like BY REFERENCE or implicit rules.

Example: Pass by Value vs. Pass by Reference

```
FUNCTION ModifyValue(NUM_VAL, NUM_REF BY REFERENCE)
  NUM_VAL := NUM_VAL + 1 // Changes copy
  NUM_REF := NUM_REF + 1 // Changes original
END FUNCTION

X := 5
Y := 10
ModifyValue(X, Y)
PRINT X // Output: 5 (X is unchanged)
PRINT Y // Output: 11 (Y is changed)
```

7. Scope of Variables

- **Local Scope:** Variables declared within a function or block are only accessible within that function/block.
- **Global Scope:** Variables declared at the top level of the program are accessible throughout the program.

Example: Global vs Local Scope

```
GLOBAL VAR := 10 // Global variable

FUNCTION TestScope()
  LOCAL VAR := 5 // Local variable
  PRINT GLOBAL_VAR // Accesses global, prints 10
  PRINT VAR // Accesses local, prints 5
  GLOBAL_VAR := GLOBAL_VAR + 1 // Modifies global VAR
  VAR := VAR + 1 // Modifies local VAR
END FUNCTION

TestScope()
PRINT GLOBAL_VAR // Prints 11
// PRINT VAR would cause an error as local VAR is out of scope
```